**NORTHERN UNIVERSITY OF BUSINESS AND TECHNOLOGY, KHULNA**

Department of Computer Science and Engineering

# AI Workflow Documentation

*Week 08 Assignment — AI Integration & Intelligent Features Development*

| | |
|---|---|
| Course: | CSE4104 — Software Development III |
| Section: | 7C |
| Team: | CSE4104-7C-T04 |
| Submitted To: | Md. Riaz Mahmud, Assistant Professor |
| GitHub: | https://github.com/0xSaikat/pluto-ai |

## WORKFLOW OVERVIEW

The Pluto AI system integrates Google Gemini 1.5 Flash for intelligent code security analysis. The workflow encompasses three core AI features: (1) vulnerability detection, (2) auto-fix generation, and (3) an interactive security assistant chat. Below is the complete workflow for each feature.

## COMPLETE AI WORKFLOW DIAGRAM

```
User Input
      ↓
Frontend (React/Vite)
      ↓
Backend API (Express.js)
      ↓
AI Service (Gemini 1.5 Flash or Local Mock AI)
      ↓
AI Response
      ↓
Backend Processing (Database Storage & Report Generation)
      ↓
Frontend Display
```

## FEATURE 1: AI-POWERED VULNERABILITY DETECTION

The following diagram illustrates the complete workflow for code vulnerability analysis:

```
Paste Code / GitHub URL
      ↓
Frontend (Analyzer.jsx)
      ↓
POST /api/scans (with JWT)
```

```
      ↓
scan.controller.js
      ↓
analyzeCodeWithAI() in ai.service.js
      ↓
Gemini 1.5 Flash REST API
      ↓
JSON response with vulnerabilities
      ↓
Backend validates & stores in DB
      ↓
Frontend renders results
```

**Step-by-step:**

1. User navigates to the Analyzer page and creates/selects a project.
2. User chooses scan type: 'Paste Code' or 'GitHub URL'.
3. User pastes source code into the textarea (or enters a GitHub URL).
4. User clicks 'Run Security Scan'.
5. Frontend sends POST /api/scans with project_id, scan_type, and source_content.
6. Backend creates a scan record with status 'running'.
7. Backend calls analyzeCodeWithAI() with the source code.
8. If GEMINI_API_KEY is set: sends structured prompt to Gemini 1.5 Flash REST API.
9. Gemini returns JSON with vulnerabilities array and risk_score.
10. Backend validates JSON, strips markdown, parses the schema.
11. Backend saves vulnerabilities to the database.
12. Backend updates scan status to 'completed' and sets risk_score.
13. Backend generates a security report automatically.
14. Frontend renders vulnerabilities with color-coded severity levels.

## FEATURE 2: AI SECURITY ASSISTANT CHAT

The interactive chat workflow:

1. User clicks the 'Ask AI Assistant' button on the Analyzer page.
2. The AIChatModal component opens, showing a welcome message from Pluto AI.
3. User types a natural-language security question in the input field.
4. User clicks 'Send'.
5. Frontend sends POST /api/ai/chat with { message, context } and JWT token.
6. Backend ai.controller.js calls chatWithAI(message, context).
7. chatWithAI() constructs a system prompt and sends it to Gemini 1.5 Flash.
8. Gemini returns a plain-text response.
9. If API fails: backend falls back to keyword-based pattern-matching responses.
10. Backend returns { success: true, reply } to the frontend.
11. Frontend appends the AI response to the chat messages array.

## FEATURE 3: AI VULNERABILITY AUTO-FIX GENERATOR

1. After a scan completes, vulnerabilities are listed on the Analyzer page.
2. Each vulnerability has an 'Auto-Fix' button.
3. User clicks the 'Auto-Fix' button on a vulnerability.
4. Frontend sends POST /api/vulnerabilities/{id}/fix.
5. Backend calls generateVulnerabilityFix(vulnerability, sourceCode).
6. If GEMINI_API_KEY is set: sends a fix-generation prompt to Gemini 1.5 Flash.
7. Gemini returns explanation, refactored code, and best practices.
8. If API fails: backend returns a hardcoded secure implementation template.
9. Backend returns the fix text to the frontend.
10. Frontend displays the fix in a modal with a formatted code block.

## FALLBACK MECHANISM

If the Gemini API key is not configured or the API call fails (network error, rate limit, timeout), the system automatically falls back to a local mock AI security engine. This ensures the application remains fully functional without interruption.

```
const analyzeCodeWithAI = async (sourceCode, scanType) => {
  if (GEMINI_API_KEY) {
    try {
      const text = await callGeminiAPI(prompt);
```

```
      return JSON.parse(text.replace(/```json/g, '').replace(/```/g,
'').trim());
    } catch (err) {
      console.warn('Gemini API failed, using local analyzer:', err.message);
    }
  }
  return mockAnalyzeCode(sourceCode, scanType);
};
```

## ERROR HANDLING SCENARIOS

| Scenario | Detection | Handling | User Experience |
|---|---|---|---|
| Empty Response | text === '' or undefined | Fallback to mock AI | Results still shown |
| Invalid JSON | JSON.parse() throws | Catch + fallback | No crash |
| Rate Limit (429) | API returns error object | Catch + fallback | No delay, offline mode |
| Network Failure | https.request error event | Catch + fallback | Seamless fallback |
| Timeout | Request timeout exceeded | Built-in timeout | Automatic fallback |
| Auth Error | 401 Unauthorized | Catch + log | Uses mock AI |

---

*Generated as part of CSE4104 Software Development III coursework at Northern University of Business and Technology, Khulna.*